FLAG-TRADER: Fusion LLM-Agent with Gradient-based Reinforcement Learning for Financial Trading

Guojun Xiong[1], Zhiyang Deng[2], Keyi Wang[3], Yupeng Cao[2], Haohang Li[2], Yangyang Yu[2], Xueqing Peng[7], Mingquan Lin[4], Kaleb E Smith[5], Xiao-Yang Liu Yanglet[3,6], Jimin Huang[7], Sophia Ananiadou[8], Qianqian Xie[7,*]

[1]Harvard University, [2]Stevens Institute of Technology, [3]Columbia University, [4]University of Minnesota, [5]NVIDIA, [6]Rensselaer Polytechnic Institute, [7]TheFinAI, [8]University of Manchester

[*]**Correspondence:** xqq.sincere@gmail.com

Abstract

Large language models (LLMs) fine-tuned on multimodal financial data have demonstrated impressive reasoning capabilities in various financial tasks. However, they often struggle with multi-step, goal-oriented scenarios in interactive financial markets, such as trading, where complex agentic approaches are required to improve decision-making. To address this, we propose FLAG-TRADER, a unified architecture integrating linguistic processing (via LLMs) with gradient-driven reinforcement learning (RL) policy optimization, in which a partially fine-tuned LLM acts as the policy network, leveraging pre-trained knowledge while adapting to the financial domain through parameter-efficient fine-tuning. Through policy gradient optimization driven by trading rewards, our framework not only enhances LLM performance in trading but also improves results on other financial-domain tasks. We present extensive empirical evidence to validate these enhancements.

1 Introduction

Algorithmic financial trading represents a critically complex decision-making domain that perpetually grapples with the intertwined challenges of synthesizing heterogeneous market signals and dynamically refining strategies (Hambly et al., 2023; Yu et al., 2024b; Li et al., 2023). Traditional reinforcement learning (RL) approaches, despite their theoretical grounding in Markov Decision Processes (MDPs), confront three fundamental limitations when deployed in financial markets. Firstly, their inability to coherently model multimodal market states—spanning sequential price movements, quantitative technical indicators, and unstructured textual sentiments—compromises data integration (Zhang et al., 2019; Nassirtoussi et al., 2014). Secondly, non-stationary data distributions inherent to financial systems systematically erode strategy generalizability across market regimes (Zhang et al., 2019). Thirdly, the heavy reliance on manually crafted technical indicators (e.g., MACD, RSI) and complex feature engineering (Liang et al., 2018) introduces subjective biases, leads to information loss, and reduces the robustness of real-time decision-making, especially in volatile market conditions.

The emergence of Large Language Models (LLMs) offer significant potential for financial decision-making by addressing key limitations of RL-based trading strategies. Leveraging their transformer architecture, they serve as multimodal feature extractors, integrating time-series and textual data, capturing long-range dependencies, and generalizing across market regimes, while also extracting nuanced sentiment signals without relying on manually crafted features (Chen et al., 2021; Yang et al., 2023a; Jin et al., 2023; Wood et al., 2021; Yu et al., 2024a; Deng et al., 2023). Nonetheless, adapting LLMs for trading presents key challenges. First, their deployment often relies on agentic frameworks (Li et al., 2024b, 2023; Yu et al., 2025), which incur high implementation and operational costs due to their complex architecture. Second, LLMs are primarily trained for static text generation, making them ill-suited for sequential decision-making in trading. This prompts us to the following question:

> *Can we design a framework that seamlessly integrates LLMs' reasoning with RL's reward-driven optimization to tackle the challenges of financial sequential decision-making?*

To resolve these interconnected challenges, we FLAG-TRADER, a unified architecture integrating linguistic processing (via LLMs) with gradient-driven RL policy optimization, as shown in Figure 1. This framework advances two synergistic innovations: a parameter-efficient fine-tuning module that jointly encodes temporal market data and tex-

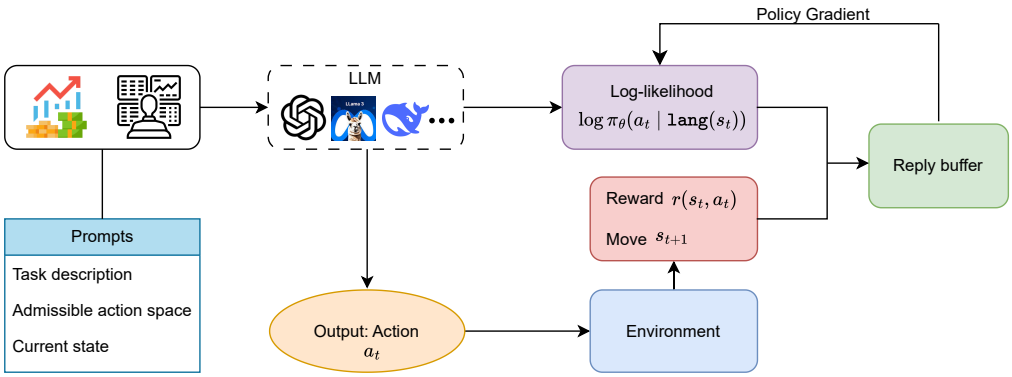

Figure 1: A high-level overview of our LLM-based reinforcement learning setup for financial trading. The environment provides the current state s_t. A prompt containing task details, the action space, and the current state is fed into the LLM, which outputs a trading action a_t. The action is executed in the environment, yielding a reward $r(s_t, a_t)$ and next state s_{t+1}. The log-likelihood $\log_{\pi_\theta}(a_t|\texttt{lang}(s_t))$ is then leveraged by a policy gradient method (e.g., PPO), with experience tuples stored in a replay buffer for iterative updates.

tual streams into unified state representations and a hybrid RL component that explicitly incorporates external environment reward gradients into policy updates, ensuring alignment with trading performance metrics. Our contributions are summarized as follows.

First, we propose the FLAG-TRADER framework, where a partially fine-tuned LLM acts as the policy network, leveraging pre-trained knowledge while adapting to the financial domain through parameter-efficient fine-tuning. The model processes market data using a textual state representation, enabling it to interpret and respond to market conditions effectively. Rather than fine-tuning the entire LLM, only a subset of its parameters is updated, balancing domain adaptation and knowledge retention. This design allows FLAG-TRADER to make informed trading decisions while remaining computationally efficient and preserving the LLM's general reasoning capabilities.

Second, we conduct extensive experiments to evaluate FLAG-TRADER across multiple financial trading tasks. Our results demonstrate that FLAG-TRADER consistently outperforms both the buy-and-hold strategy and LLM-agentic baselines, particularly in terms of cumulative return and Sharpe ratio, which we prioritize for financial performance assessment. Notably, our approach enables a small-scale (135M parameter) open-source LLM to surpass much larger proprietary models, highlighting the effectiveness of RL fine-tuning in optimizing LLM-driven trading strategies. These findings underscore the potential of integrating LLMs with RL to enhance financial decision-making while maintaining computational efficiency.

2 Related Work

RL in Finance. RL has shown promise for financial decision-making, spanning Q-learning approaches for Sharpe ratio maximization (Gao and Chan, 2000), dynamic asset allocation (Jangmin et al., 2006), deep Q-learning (Jeong and Kim, 2019), tabular SARSA (de Oliveira et al., 2020), policy-based portfolio optimization (Shi et al., 2019), and actor-critic methods (Ye et al., 2020) enhanced by adversarial training (Liang et al., 2018) and transformer-based architectures (Huang et al., 2024). Recent research efforts in RL for financial applications have been greatly aided by open-source frameworks like FinRL (Liu et al., 2022), which standardize implementations and provide reproducible benchmarks. Comprehensive surveys (Hambly et al., 2023; Sun et al., 2023) further showcase advances in both methodological rigor and real-world deployment. Despite these advances, RL-based trading still requires large training data, struggles with non-stationary markets, and faces challenges incorporating multimodal information in real time.

LLMs in Finance. A growing trend is the integration of LLMs into financial decision-making. Hybrid systems like FinCon (Yu et al., 2025) and TradingGPT (Li et al., 2023) leverage language understanding to enhance trading agents, while domain-specific models such as FINBERT (Araci, 2019; Yang et al., 2020), FLANG (Shah et al.,

2022) and OPEN-FINLLMS (Xie et al., 2024b) have excelled at financial text tasks through specialized pre-training. Recent efforts include machine reading comprehension (Zhang and Zhang, 2023), open-source financial LLMs (Liu et al., 2023), BloombergGPT with domain-adapted tokenization (Wu et al., 2023), and InvestLM (Yang et al., 2023b) featuring numerical reasoning—achieving strong results in sentiment analysis (Huang et al., 2023), earnings call interpretation (Xie et al., 2023), and regulatory document processing. Additionally, FINBEN (Xie et al., 2024a), benchmark study for LLMs in finance, have emerged to comprehensively evaluate model performance across various financial tasks. However, LLM-based methods often lack sequential decision-making mechanisms, are computationally expensive (especially with RL), and struggle with non-stationary market conditions.

LLM Agents for Sequential Decision Making. The integration of LLMs with agentic frameworks has opened new avenues for financial decision-making. For instance, FINMEM (Yu et al., 2024a) introduced memory-augmented LLM agents for portfolio management, FINAGENT (Zhang et al., 2024) leveraged hierarchical structures in high-frequency trading, and multi-agent systems like FINROBOT (Yang et al., 2024) and FINCON (Yu et al., 2024b) emphasize contextual adaptation and collaboration. Meanwhile, fine-tuning LLMs and vision-language models (VLMs) with reinforcement learning has proven effective in complex tasks: LLaRP (Szot et al., 2023) positions LLMs as generalizable policies for embodied tasks, and RL-tuned VLMs (Zhai et al., 2024) enhance multi-step decision-making. However, LLMs remain computationally expensive for real-time deployment, and risk-sensitive trading demands robustness to non-stationary markets, calling for careful model complexity and balanced exploration-exploitation.

3 Problem Statement

We define the financial decision-making process as a finite horizon partially observable Markov decision process (MDP) with time index $\{0, \cdots, T\}$, represented by the tuple: $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$, where each component is described in detail below.

State. The state space $\mathcal{S} = \mathcal{X} \times \mathcal{Y}$ consists of two components: market observations and trading account balance, i.e., $s_t = (m_t, b_t) \in \mathcal{S}$. Specifically, $m_t = (P_t, N_t) \in \mathcal{X}$ represents the *market observation process*, includingstock price P_t at time t, and financial news sentiment or macroeconomic indicators N_t; $b_t = (C_t, H_t) \in \mathcal{Y}$ represents the *trading account balance*, including available cash C_t at time t, and number of stock shares H_t.

Action. The agent chooses from a discrete set of trading actions $\mathcal{A} = \{\texttt{Sell} : -1, \texttt{Hold} : 0, \texttt{Buy} : 1\}$, where $a_t = -1$ denotes selling all holdings (liquidate the portfolio), $a_t = 0$ denotes holding (no trading action), and $a_t = 1$ represents buying with all available cash (convert all cash into stocks).

State Transition. The state transition dynamics are governed by a stochastic process $s_{t+1} \sim \mathcal{T}(\cdot|s_t, a_t)$. The trading account evolves according to the following equations:

- If `Sell`: $C_{t+1} = C_t + H_t P_{t+1}, \quad H_{t+1} = 0.$
- If `Hold`: $C_{t+1} = C_t, \quad H_{t+1} = H_t.$
- If `Buy`: $C_{t+1} = 0, \quad H_{t+1} = H_t + \frac{C_t}{P_{t+1}}.$

Reward. The agent receives a reward based on the daily trading profit & loss (PnLs):

$$R(s_t, a_t) = SR_t - SR_{t-1},$$

where SR_t denotes the Sharpe ratio at day t, computed by using the historical PnL from time 0 to time t. Moreover, PnL at time t is calculated as

$$pnl_t := (C_t - C_{t-1}) + (H_t P_t - H_{t-1} P_{t-1}).$$

Then, the Sharpe ratio SR_t at time t can be calculated as:

$$SR_t := \frac{\mathbb{E}[pnl_1, \cdots, pnl_t] - r_f}{\sigma[pnl_1, \cdots, pnl_t]}, \tag{1}$$

where $\mathbb{E}[pnl_1, \cdots, pnl_t]$ is the sample average of daily PnL up to time t, r_f is the risk-free rate, and $\sigma[pnl_1, \cdots, pnl_t]$ is the sample standard deviation of daily PnL up to time t.

The goal is to find an admissible policy π to maximize the expected value of cumulative discounted reward, i.e.,

$$\max_{\pi} V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{s_0 = s, a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim \mathcal{T}(\cdot|s_t, a_t)}} \left[\sum_{t=0}^{T} \gamma^t R_t \right], \tag{2}$$

where R_t is a shortened version $R(s_t, a_t)$ and $\gamma \in (0, 1]$ is the discount factor controlling the importance of future rewards.

Our goal is to train an LLM agent parameterized by θ to find the optimized policy π_θ for (2), i.e.,

$$a_t \sim \pi_\theta(\cdot|s_t) = \text{LLM}(\texttt{lang}(s_t); \theta), \tag{3}$$

where $\texttt{lang}(s_t)$ are the prompts generated by converting state s_t into structured text. The proposed pipeline is illustrated in Figure 1.